Database: NoSQL (MongoDB)

Section A — Concept Application

S1 – MongoDB is well suited to QuickEats because its document-oriented model provides greater flexibility for rapidly changing application requirements.

- Schema flexibility: MongoDB does not require every document to have the same fixed schema. New fields such as isVeg, preparationTime, or addOns can be added to menu-item documents without altering a table or migrating all existing rows.
- Data nesting: Related data can be embedded in one document. For example, a restaurant document can contain a menu array, with each menu item containing its price, isVeg, preparation time, and add-ons. This can reduce the need for joins.
- Horizontal scalability: MongoDB supports sharding, which distributes data across multiple servers. This allows the system to scale by adding servers as the number of customers, restaurants, and orders increases.
- Trade-off: MongoDB's flexible schema can result in inconsistent document structures if validation is not properly implemented. Complex relationships and transactions can also sometimes be easier to manage with a relational database such as MySQL.

Therefore, MongoDB is attractive for a rapidly evolving food-delivery application because it combines flexible schemas, nested data, and horizontal scalability.

S2 – **Create the database and collection**

```
use quickeats_db

db.createCollection("orders")
```

Insert one order using insertOne:

```
db.orders.insertOne({
    customerName: "Rahul Sharma",
    items: [
        {
            name: "Paneer Pizza",
            quantity: 2,
            price: 299
        },
        {
            name: "Garlic Bread",
            quantity: 1,
            price: 149
        }
```

```
    ],
    totalAmount: 747,
    deliveryAddress: "Alkapuri, Vadodara",
    status: "pending"
})
```

Insert three orders using insertMany:

```
db.orders.insertMany([
    {
        customerName: "Priya Patel",
        items: [
            {
                name: "Veg Burger",
                quantity: 2,
                price: 199
            }
        ],
        totalAmount: 398,
        deliveryAddress: "Manjalpur, Vadodara",
        status: "confirmed"
    },
    {
        customerName: "Amit Shah",
        items: [
            {
                name: "Biryani",
                quantity: 2,
                price: 250
            },
            {
                name: "Cold Drink",
                quantity: 1,
                price: 60
            }
        ],
        totalAmount: 560,
        deliveryAddress: "Gotri, Vadodara",
        status: "pending"
    },
    {
        customerName: "Neha Mehta",
        items: [
            {
                name: "Margherita Pizza",
                quantity: 1,
                price: 350
            },
```

```
            {
                name: "Pasta",
                quantity: 1,
                price: 280
            }
        ],
        totalAmount: 630,
        deliveryAddress: "Sayajigunj, Vadodara",
        status: "confirmed"
    }
])
```

S3 –

Assuming the specific customer's name is "Rahul Sharma":

```
db.orders.find({
    customerName: { $eq: "Rahul Sharma" },
    status: { $in: ["pending", "confirmed"] },
    totalAmount: { $gt: 500 }
})
```

Query with projection, sorting, and limiting:

```
db.orders.find(
    {
        customerName: { $eq: "Rahul Sharma" },
        status: { $in: ["pending", "confirmed"] },
        totalAmount: { $gt: 500 }
    },
    {
        _id: 0,
        customerName: 1,
        totalAmount: 1,
        status: 1
    }
).sort({
    totalAmount: -1
}).limit(10)
```

S4 –

Difference between the operators

- **$**set creates a field or changes its existing value.
- $push adds a new value to an array. It allows duplicate values.

- $addToSet adds a value to an array only when that value does not already exist, preventing duplicates.

Required update:

```
db.orders.updateOne(
    { _id: 101 },
    {
        $set: {
            status: "confirmed",
            restaurantNote: "Order accepted"
        },
        $push: {
            statusHistory: {
                event: "confirmed",
                time: new Date()
            }
        }
    }
)
```

S5 –

Assuming the order documents contain restaurantId, totalAmount, status, and orderDate:

```
db.orders.aggregate([
    {
        $match: {
            status: "completed",
            orderDate: {
                $gte: new Date(Date.now() - 7 * 24 * 60 * 60 * 1000)
            }
        }
    },
    {
        $group: {
            _id: "$restaurantId",
            totalRevenue: {
                $sum: "$totalAmount"
            },
            completedOrders: {
                $sum: 1
            }
        }
    },
    {
        $project: {
            _id: 0,
            restaurantId: "$_id",
```

```
            totalRevenue: 1,
            completedOrders: 1
        }
    },
    {
        $sort: {
            totalRevenue: -1
        }
    }
])
```

Explanation

1. $match filters the collection to completed orders from the last seven days.
2. $group groups orders by restaurant and calculates total revenue with $sum. It also counts the completed orders.
3. $project formats the output and removes the default _id field.
4. $sort ranks restaurants from highest to lowest total revenue.

Aggregation is more efficient than loading all orders into Node.js because MongoDB performs the filtering, grouping, and calculations close to the data. This reduces network transfer, application memory usage, and unnecessary processing in the Node.js application.

S6 –

Compound indexes in MongoDB:

An index is a data structure MongoDB maintains to locate documents more efficiently. Without a suitable index, MongoDB may need to scan a large portion or all of the collection to find matching documents. With an appropriate index, it can use an index scan (IXSCAN) to locate matching records much faster.

For the QuickEats query filtering by both customerId and status, create this compound index:

```
db.orders.createIndex({
    customerId: 1,
    status: 1
})
```

A query such as:

```
db.orders.find({
    customerId: 101,
    status: "pending"
```

```
})
```

Trade-off

Indexes consume **disk space and memory**, and MongoDB must update the relevant indexes whenever documents are inserted, updated, or deleted. Therefore, having too many unnecessary indexes can slow write operations and increase storage requirements.

However, the index is clearly worth the cost when:

- The collection is large.
- A query is executed frequently.
- The query filters or sorts on the indexed fields.
- The query currently examines a large number of documents.
- Faster read performance is more important than the relatively small additional write overhead.

In this scenario, with **500,000 documents and queries taking over two seconds**, a compound index on {customerId: 1, status: 1} is likely worthwhile, especially if customer/status searches are frequent. The actual benefit should be confirmed with explain("executionStats").